Ultimate Amiga OS

Technical Reference Manual

Version 0.1.0-dev

June 20, 2026

UAOS is a standalone, bare-metal 64-bit operating system for x86_64 platforms.
It is derived from the AROS open-source Amiga replacement kernel and features
transparent M68k JIT emulation and native GNU tool integration.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Project Overview

Ultimate Amiga OS (UAOS) is a standalone bare-metal x86_64 operating system designed to provide a native Amiga-compatible environment on modern hardware without requiring any host operating system. It achieves this through:

- An extended AROS microkernel providing Exec, DOS, and Intuition semantics on 64-bit hardware.
- A transparent M68k JIT emulation sandbox that executes legacy Amiga "Hunk" binaries inside a 4 GB virtual address space.
- An illegal-opcode trap mechanism (`ILLEGAL`/`0x4AFC`) that intercepts system calls at the ROM vector level and dispatches them to native 64-bit C implementations.
- An MMU sandbox using 2 MB huge pages with hardware-register windows deliberately left non-present to transparently virtualise custom chipset I/O via #PF exceptions.

1.2 Architectural Constraints

1. **No Linux kernel.** UAOS boots directly into its own x86_64 AROS-derived microkernel. No Linux, no POSIX kernel layer.
2. **Trap-based thunking.** M68k Hunk binaries are loaded into the 4 GB sandbox. System calls are intercepted via `ILLEGAL + $414D` magic-token sequences routed to native 64-bit AROS libraries.
3. **MMU hardware virtualisation.** x86_64 page tables map the sandbox with 2 MB huge pages. The Amiga hardware register window `0x00B00000`–`0x00DFFFFF` is marked NOT PRESENT.
4. **Coexisting filesystem namespace.** `dos.library` handlers resolve Amiga assigns (`SYS:`, `S:`, `C:`, `RAM:`) alongside POSIX translation paths.
5. **AROS Kickstart firmware.** The open-source AROS replacement ROM is used as the modular firmware base with vectors patched at build time.

Chapter 2

System Architecture

2.1 Boot Sequence

1. GRUB2 loads the ELF64 kernel via the Multiboot2 protocol.
2. `uaos_kernel_entry.asm` transitions the CPU from 32-bit protected mode into 64-bit long mode, sets up a bootstrap identity-map page table, and calls `uaos_kernel_main()`.
3. The C kernel entry point validates the Multiboot2 magic value, initialises the VGA text-mode console and UART, then calls:
   - `APIC_Init()` — configures the local APIC for ExtINT delivery.
   - `FB_Init()` — parses the Multiboot2 framebuffer tag.
   - `IDT_Init()` — installs 256-vector IDT and remaps 8259A PIC.
   - `UAOS_MMU_Init()` — installs the sandbox page tables.
   - `RTC_Init()` — CMOS real-time clock with UIE interrupt.
   - `VFS_Init()` — virtual filesystem layer and RAM: mount.
   - `BlockDev_Init()` — block device layer.
   - `virtio_blk_init()` — VirtIO block device detection.
   - `ide_init()` — IDE/ATAPI controller.
   - `UAOS_ROM_RegisterAll()` — populates the ROM module registry.
   - `UAOS_Bridge_Init()` — allocates the 4 GB guest RAM window.
   - `Task_Init()` — Ring-3 task scheduler and TSS.
   - `PS2Mouse_Init()` and `PS2Kbd_Init()` — PS/2 input drivers.
   - `net_stack_init()` — TCP/IP stack and NIC auto-probe.
   - `Desktop_Draw()` and `ShellWin_Init()` — graphical shell.
   - `ShellWin_RunStartupSequence()` — executes `S:Startup-Sequence`.
4. Enters the `hlt`-loop event dispatcher.

2.2 M68k Thunk Dispatch

The thunk mechanism replaces selected Exec jump-table vectors in the AROS ROM with 8-byte breakout stubs assembled by the `MK_THUNK` macro in `emulation/rom_patches/rom_traps.s`:

Listing 2.1: MK_THUNK macro

```
MK_THUNK MACRO
        ILLEGAL             ; $4AFC --- triggers JIT breakout
        dc.w    \1          ; UAOS magic token ($414D = "AM")
        dc.w    \2          ; function index (1-based)
        rts                 ; return to guest after dispatch
        ENDM
```

When the JIT core decodes `0x4AFC` it invokes `UAOS_Bridge_IllegalOpcode()`. The C dispatcher:

1. Reads the two words immediately following the ILLEGAL opcode.
2. Validates the `$414D` signature.
3. Extracts arguments from the M68k register state (A1, D0, D1, ...).
4. Calls the native 64-bit stub.
5. Writes return values back into D0/A0.
6. Advances the guest PC by 6 bytes.

2.3 MMU Sandbox

`kernel/exec/mmu_sandbox.c` constructs four-level x86_64 paging tables mapping the entire 4 GB guest address space using 2 MB huge pages. Table 2.1 shows the critical address assignments.

Address Range	Size	Description
0x00000000-0x001FFFFF	2 MB	Chip RAM
0x00200000-0x009FFFFF	8 MB	Fast RAM (lower)
0x00A00000-0x00AFFFFF	1 MB	Slow RAM / ranger
0x00B00000-0x00DFFFFF	3 MB	**Custom chip registers (NOT PRESENT)**
0x00E00000-0x00EFFFFF	1 MB	Extended ROM / card space
0x00F80000-0x00FFFFFF	512 KB	Kickstart ROM mirror
0x01000000-0x7FFFFFFF	2 GB	Extended Fast RAM
0x80000000-0xFFFFFFFF	2 GB	x86_64 kernel space

Table 2.1: UAOS Guest Address Space Map

Pages in the hardware register window have `PAGE_PRESENT` cleared and `PAGE_NO_CACHE` set. Any access faults to this region are caught by the #PF handler in `kernel/exec/page_fault_handler.c` and forwarded to the chip emulator.

2.4 Ring-3 Userspace and System Calls (`INT 0x80`)

UAOS implements a Ring-3 (user-mode) task execution model for native 64-bit ELF binaries. These programs are compiled with `-ffreestanding -nostdlib -fPIE -pie` and linked against `uaos_start.o` to produce position-independent binaries wrapped in a 32-byte `UAOS` header (type `0x0003` / `UAOS_TYPE_X64`).

1. **GDT & TSS Setup** — The ELF64 loader in `task.c` sets up the task stack, GDT user code/data segments (`SEG_USER_CS`, `SEG_USER_DS`), and registers the task with the Task State Segment (TSS). Transition to Ring 3 is performed via an `iretq` sequence targeting the binary's entry point.

2. **INT 0x80 System Calls** — Userspace utilities interact with the kernel using software interrupts via the `INT 0x80` vector. Arguments are passed via registers:

   - **RAX** — System call number (e.g. `0x02` for write, `0x0C` for getcwd).
   - **RDI, RSI, RDX** — Arguments 1, 2, 3.
   - **RAX** — Return value (negative values represent errors).

   The dispatcher in `syscall_dispatch.c` implements page allocation (`sys_alloc`), process management (`sys_spawn`, `sys_wait`, `sys_exit`), VFS file/directory interfaces (`sys_getcwd`, `sys_opendir`, `sys_readdir`, `sys_closedir`, `sys_stat`), and GUI windowing syscalls (`create_window`, `destroy_window`, `draw_text`, `draw_rect`, `present`, `get_event`, `set_scroll_info`, `set_scroll`).

3. **Current Userspace Programs** — `hello`, `pwd`, `file`, `strings`, `find`, and `Guide` are built from `system/userspace/` and staged into `C:` (or `Tools:` for `Guide`).

4. **Execution State & Redirection** —

   - **Per-Task CWD** — Each task struct tracks its own current working directory in `task_cwd` which VFS resolves relative paths against.
   - **Output Buffering** — Stdout writes are buffered per-task in `task_out` and flushed line-by-line via `native_print_fn` to the active shell window.
   - **Synchronous Exit** — When a child task exits via `sys_exit`, the parent task is signaled with `SIGF_CHILD`, allowing the command-line interface to wait for command completion.

Chapter 3

Kernel Subsystems

3.1 ROM Module Registry (`rom_modules.c`)

`UAOS_ROM_RegisterAll()` is called once at boot. It populates a static array of `UaosRomModule` descriptors, each holding:

- Library name string (e.g. `"exec.library"`).
- Library version number.
- 32-bit Amiga base address.
- Array of native function pointers (indexed 1-based to match `rom_traps.s` function indices).

3.1.1 Registered ROM Modules

Table 3.1 lists the modules registered by `UAOS_ROM_RegisterAll()`.

Module	Version	Base	Purpose
`exec.library`	45	0x00000004	Process/task control, memory, signals
`utility.library`	37	0x00000050	String utilities
`console.device`	40	0x00000060	Console I/O
`mathffp.library`	40	0x00000060	Soft-float arithmetic
`locale.library`	38	0x000000B0	Localisation
`ixemul.library`	53	0x00000090	Unix compatibility layer
`timer.device`	40	0x000000A0	Timing (RTC-backed)
`keyboard.device`	40	0x000000B0	Keyboard input (PS/2)
`graphics.library`	40	0x000000C0	Graphics primitives
`dos.library`	40	0x000000D0	VFS file system operations
`bsdsocket.library`	4	0x00003000	BSD socket API
`workbench.library`	45	0x00000000	Workbench integration
`intuition.library`	40	0x00005000	GUI/Intuition API

Table 3.1: ROM modules registered at boot

3.2 Thunk Handler (`thunk_handler.c`)

The `UAOS_AMIGA_TO_HOST()` macro translates any 32-bit Amiga guest address to the corresponding host linear address:

Listing 3.1: Address translation macro

```
#define UAOS_AMIGA_TO_HOST(amiga_addr) \
    ((void *)((uintptr_t)(uaos_ram_base) + (uint32_t)(amiga_addr)))
```

3.3 Page Fault Handler (page_fault_handler.c)

The ISR entry stub `uaos_page_fault_isr` (written in inline GAS assembly) saves all 15 GPRs onto the kernel stack, computes pointers to the `InterruptFrame` and `SavedRegs` blocks, and calls the C function `UAOS_PageFaultHandler()`. On return from the C handler, GPRs are restored and `IRETQ` resumes execution past the faulting instruction.

3.4 Native Command System (kernel/shell/)

The shell dispatches `C:` binaries through a static registry in `native_cmd.c`. `k_native_cmds[]` maps a lowercase command name to a `NativeCmdFn` handler and an optional AmigaDOS-style template string. Key components:

- `native_cmd.c/h` — registry and case-insensitive lookup.
- `cmd_template.c/h` — AmigaDOS-style argument parser (`/A`, `/K`, `/S`, `/N`, `/M`, `/F`).
- `resident_cmd.c/h` — in-memory resident command cache (256 KB, up to 16 slots).
- `cmd_*.c` — individual command implementations (65+ commands).

3.5 Virtual Filesystem Layer (vfs.c, ramfs.c)

UAOS provides a thin VFS dispatch layer over an in-memory RAM filesystem. Paths use AmigaDOS format: `VOL:path/to/file`.

Listing 3.2: VFS file handle

```
typedef struct {
    RamFsNode *node;     /* NULL = invalid / not open */
    uint32_t   pos;      /* current read/write position */
} VfsFile;
```

RAM Filesystem (`ramfs.c`): BSS-backed storage with up to 1024 nodes, 512 KB per file, and a 4 MB data pool. Auto-mounted at boot as `RAM:` with `T`, `ENV`, `CLIPS`, and `S` directories.

Filesystem drivers: FAT32 (mount, open, seek, size, format; cluster read/write pending), PFS3 (mount), EXT4 (read-only mount), and ISO9660 (CD-ROM reader that loads files into RAMFS at boot).

Partition volumes: The block device layer detects MBR partitions on VirtIO/IDE devices and registers them as child block devices (e.g. `virtio01`). Formatted partitions are auto-mounted in the VFS by their display name (e.g. `DH0:`) and by their FAT32 volume label (e.g. `WORK:`).

VFS Operations include `VFS_Open`, `VFS_Read`, `VFS_Write`, `VFS_MkDir`, `VFS_Delete`, `VFS_ResolveDir`, `VFS_MountPartition`, `VFS_GetMountCount`, and `VFS_GetMountName`.

3.6 Block Device Layer (blockdev.c)

Unified interface for storage devices. Key operations:

- `BlockDev_Register(dev)` / `BlockDev_Unregister(dev)`

- `BlockDev_RegisterPartition(parent, idx, start, count, name)` — creates child partition devices from MBR entries.
- `BlockDev_CheckFormatted(dev)` — reads boot sector for valid FAT BPB signature.
- `BlockDev_ReadVolLabel(dev, buf, max)` — extracts the 11-byte FAT32 volume label from boot sector offset 71.
- `BlockDev_Read` / `BlockDev_Write` / `BlockDev_GetCapacity`

VirtIO Block Driver (`virtio_blk.c`): PCI scanning for vendor 0x1AF4 / device 0x1001, capacity reporting from configuration space. Virtqueue I/O is pending implementation.

Chapter 4

Build System

4.1 Prerequisites

Listing 4.1: Install build dependencies (Ubuntu/Debian)

```
sudo apt install nasm gcc binutils xorriso \
    grub-pc-bin grub-common \
    binutils-m68k-linux-gnu python3
```

4.2 Building the ISO

Listing 4.2: Full build pipeline

```
# Single command builds everything:
./scripts/build_iso.sh

# Clean rebuild:
./scripts/build_iso.sh --clean
```

The script compiles the NASM and C kernel sources, links the ELF64 kernel, embeds any M68k binaries in `emulation/binaries/`, builds native x86-64 Ring-3 userspace programs from `system/userspace/`, stages the `system/` skeleton into a dynamic `SYS_ROOT`, and invokes `grub-mkrescue` to produce `build/Ultimate_Amiga_OS.iso`.

4.3 Testing in QEMU

Copy the OVMF variables file once (required for UEFI variable store):

Listing 4.3: One-time OVMF setup

```
cp /usr/share/OVMF/OVMF_VARS_4M.fd /tmp/ovmf_vars.fd
```

Listing 4.4: QEMU test invocation

```
qemu-system-x86_64 \
  -machine q35,usb=off \
  -drive if=pflash,format=raw,readonly=on,\
file=/usr/share/OVMF/OVMF_CODE_4M.fd \
  -drive if=pflash,format=raw,file=/tmp/ovmf_vars.fd \
  -device piix3-ide,id=ide \
  -drive if=none,id=cdrom,media=cdrom,\
file=build/Ultimate_Amiga_OS.iso \
```

```
  -device ide-cd,drive=cdrom,bus=ide.0 \
  -m 512M -vga virtio -no-reboot -no-shutdown \
  -serial stdio
```

The `usb=off` flag is required to prevent QEMU's USB tablet device from conflicting with the PS/2 relative mouse driver. The explicit `piix3-ide` controller is needed because the Q35 chipset has no built-in IDE controller. Add `-serial stdio` to see kernel debug output on the terminal.

Chapter 5

Graphics and Window Manager

5.1 Framebuffer Initialisation

GRUB2 fills the Multiboot2 framebuffer tag (type 8) from the UEFI GOP driver. `FB_Init()` parses this tag to populate the global `FbState g_fb` structure, which holds the physical base address, pixel dimensions, pitch (bytes per row), and bit depth (24 or 32 bpp).

All drawing primitives (`FB_FillRect`, `FB_DrawHLine`, `FB_DrawVLine`, `FB_PutPixel`, `FB_PutStr`) clip coordinates to the framebuffer bounds before writing, supporting windows that extend partially off-screen.

5.2 Desktop

`Desktop_Draw()` renders the full Workbench-style backdrop:

- **Menu bar** (top, 20 px) — Workbench menu labels and clock area.
- **Backdrop** — light grey base with a dark-grey stipple dot overlay, one dot per two pixels alternating rows/columns (classic Amiga pattern).
- **Disk icons** — top-right corner, stacked vertically.
- **Information window** — centred boot status window.
- **Status bar** (bottom, 18 px) — kernel idle message and RAM size.

`Desktop_RedrawRect(rx, ry, rw, rh)` repaints a sub-rectangle of the desktop (backdrop + overlays) without a full-screen repaint. This is used by the window manager to erase the old window footprint during drag and resize, avoiding the full-screen flicker that would otherwise occur.

5.3 Window Manager

`kernel/display/wm.c` implements a lightweight window manager supporting up to eight windows simultaneously.

5.3.1 Data Structures

Listing 5.1: WmWindow structure

```
1 typedef struct {
2     int       x, y, w, h;
```

```
    char       title[32];
    WM_DrawFn  draw;      /* repaint callback: (x, y, w, h) */
    WM_KeyFn   on_key;    /* keystroke callback: (char c)   */
    int        active;
} WmWindow;
```

5.3.2 Z-Order and Focus

Windows are stored in `g_wins[]` and indexed by a separate `g_zorder[]` array (`[0]` = back, `[g_nwins-1]` = top). `WM_Redraw()` paints windows back-to-front. Clicking a window calls `raise_window()` to promote it to the top of the z-stack.

5.3.3 Drag and Resize

- **Drag** — a mouse-down on the title bar records the grab offset. Each subsequent mouse move erases the old footprint with `Desktop_RedrawRect`, updates `w->x/y`, and repaints all windows.
- **Resize** — a mouse-down on the 16×16 resize grip in the bottom-right corner records the base size and origin. Mouse movement computes the new dimensions relative to the drag start point.
- Windows may extend off the left, right, and bottom screen edges. The title bar is always constrained to remain on-screen.

5.4 Software Cursor

`kernel/display/cursor.c` implements a 16×16 Amiga-style arrow sprite cursor. The cursor is rendered by saving the background pixels beneath it, drawing the sprite, and on each move restoring the saved background before re-saving and re-drawing at the new position. This ensures the desktop and window content are never permanently overwritten by cursor movement.

5.5 Additional Windows

Besides the main shell and Workbench, the kernel provides several standalone windows:

- **About UAOS** — `kernel/display/about_win.c`
- **Calculator** — `kernel/display/calc_win.c`, opened by `calculator`
- **Clock** — `kernel/display/clock_win.c`, opened by `clock`
- **NetInfo** — `kernel/display/netinfo_win.c`, opened by `netinfo`
- **Pointer preferences** — `kernel/display/pointer_prefs.c`, opened by `pointer`
- **User window / Guide viewer** — `kernel/display/user_window.c`, opened by `Guide`
- **VIM editor** — `kernel/display/vim_win.c`, opened by `vim`
- **File browser** — `kernel/display/filebrowser.c`, opened from Workbench icons

Chapter 6

Shell and Built-in Commands

The shell window (`shell_win.c`) registers with the window manager via `WM_AddWindow()` and implements a scrollable terminal with up to 128 lines of history and an input line with backspace support.

6.1 Command Execution Flow

The shell resolves a command in the following order:

1. **Shell built-ins** — implemented directly in the shell.
2. **Resident commands** — cached in memory by `resident`.
3. **Native C: commands** — dispatched from `k_native_cmds[]`.
4. **M68k Hunk binaries** — loaded into the Musashi emulator.
5. **Ring-3 userspace ELF64 binaries** — run with `INT 0x80` syscalls.

All command lookup is case-insensitive. Native commands may declare an AmigaDOS-style template (`/A` required, `/K` keyword, `/S` switch, `/N` numeric, `/M` multiple, `/F` free-form) that is parsed automatically before the handler is invoked.

6.2 Built-in Commands

Command	Description
`help`	List available commands
`cd [path]`	Change or show current directory
`alias [name cmd]`	Create or list aliases
`unalias <name>`	Remove an alias
`set [name val]`	Set or list local variables
`unset <name>`	Remove a local variable
`path [dirs...]`	Show or set command search path
`setenv <name> <value>`	Set a global variable
`unsetenv <name>`	Remove a global variable
`showconfig`	Show hardware configuration

Table 6.1: UAOS Shell Built-in Commands

6.3 Native C: Commands

6.4 Ring-3 Userspace Commands

6.5 Filesystem Operations

The shell integrates with the VFS layer for all filesystem operations. Partition volumes (e.g. `DH0:`, `WORK:`) can be accessed with `cd` and `dir` once auto-mounted at boot or after formatting with the `format` command.

Command	Description
`version`	Show OS version and architecture
`mem`	Display memory information
`libs`	List loaded ROM libraries
`clear`	Clear shell history
`reboot`	Warm reboot
`dir [path]`	List directory contents
`makedir <path>`	Create a directory
`delete <path>`	Delete a file or empty directory
`type <file>`	Display file contents
`copy <src> <dst>`	Copy a file
`rename <from> <to>`	Rename or move a file
`echo <text>`	Print text
`protect <flags> <path>`	Set attributes (±r, ±h)
`attr <path>`	Show file attributes
`info [device]`	Show mounted disks and volumes
`date`	Show date and time
`which <cmd>`	Locate a command
`disks`	List block devices
`fdisk <device>`	Partition a block device
`format <dev> [fs]`	Format a partition (FAT32)
`pointer`	Open pointer preferences
`run <cmd> [args]`	Run a command in a new CLI
`assign [name: target]`	Create or list assigns
`execute <script>`	Run a script file
`loadwb`	Launch the Workbench desktop
`calculator`	Open calculator window
`clock`	Open clock window
`ifconfig`	Show or configure network
`ping`	Send ICMP echo requests
`route`	Show routing table and ARP cache
`nslookup`	Resolve a hostname
`ntpd`	Synchronise time via NTP
`netstart / netstop`	Start or stop the network stack
`netinfo`	Open network info window
`grep / more / list / search / sort / join`	Text/file processing
`vim`	Inline text editor
`newcli / newshell`	Open a new shell window
`ask / prompt / why / failat / quit / endcli`	Shell scripting
`resident`	Manage resident commands
`ps / jobs / wait / changetaskpri`	Process/task control
`filenote / relabel / install / diskchange / addbuffers`	Disk maintenance
`requestchoice / requestfile`	Dialogs
`status / avail / stack`	System utilities
`getenv / unset`	Environment access

Table 6.2: UAOS Native C: Commands

Command	Description
`pwd`	Print working directory
`find [path] [-name pat] [-type f|d]`	Recursively search directories
`file <path>...`	Identify file format from magic numbers
`strings <path>... [-n minlen]`	Extract printable strings
`hello`	Print a simple greeting
`Guide`	AmigaGuide help viewer

Table 6.3: UAOS Ring-3 Userspace Commands

Chapter 7

TCP/IP Networking

UAOS includes a complete freestanding TCP/IP stack. It auto-detects the network card at boot, attempts DHCP, and falls back to a static IP address if DHCP times out. The stack runs entirely in the kernel event loop — no threads or blocking system calls are required.

7.1 Architecture

```
Shell / bsdsocket.library
        |
        v
 icmp / tcp / udp   (kernel/net/)
        |
        v
  ip.c  <-->  arp.c
        |
        v
 NetDevice  (abstract hardware interface)
        |
   +----+----+
   |         |
   v         v
virtio_net  e1000
(QEMU)   (VirtualBox)
```

7.2 Network Hardware Drivers

7.2.1 NetDevice Abstraction (`net_device.h`)

All stack components call through the `NetDevice` interface rather than any specific driver. `netdev_probe()` (called by `net_stack_init_ex()`) scans PCI for a supported device, registers the first one found, and calls its `init()` function. The e1000 is tried before virtio-net so that VirtualBox guests are served without configuration.

```
typedef struct NetDevice {
    const char *name;
    int  (*init)(struct NetDevice *dev);
    void (*get_mac)(struct NetDevice *dev, uint8_t *buf);
    int  (*send)(struct NetDevice *dev, const uint8_t *data, uint16_t
       len);
    void (*poll)(struct NetDevice *dev);
```

```
    void (*set_rx_callback)(struct NetDevice *dev, netdev_rx_fn cb);
    void (*setup_irq)(struct NetDevice *dev);
    void *priv;
} NetDevice;
```

7.2.2 VirtIO-Net (`virtio_net.c`)

- PCI vendor `0x1AF4`, device `0x1000` (legacy VirtIO 0.9) or `0x1041` (modern).
- I/O-port or MMIO BAR0; split virtqueue descriptor rings.
- MSI-X capability is present but kept disabled; interrupts use the 8259A PIC (IRQ 10 on Q35 QEMU).
- Used by: QEMU `-device virtio-net-pci`.

7.2.3 Intel 82540EM e1000 (`e1000.c`)

- PCI vendor `0x8086`, device `0x100E` (and related 8254x family IDs).
- 128 KB MMIO BAR0; legacy 16-byte TX/RX descriptor rings (32 slots each).
- MAC address read from EEPROM via the EERD register at initialisation.
- IRQ delivered via ICR (self-clearing); routed through the 8259A PIC.
- Used by: VirtualBox (Intel PRO/1000 MT Desktop) and QEMU `-device e1000`.

7.3 Stack Initialisation

Call `net_stack_init()` once from `uaos_kernel_main()`:

```
/* DHCP with 1-second timeout, static fallback */
net_stack_init(IPV4(10,0,2,15), IPV4(10,0,2,2), IPV4(255,255,255,0));

/* Explicit DHCP timeout (ms) */
net_stack_init_ex(IPV4(10,0,2,15), IPV4(10,0,2,2),
                  IPV4(255,255,255,0), 2000);
```

The `IPV4(a,b,c,d)` macro builds a host-byte-order `ipv4_t`:

```
#define IPV4(a,b,c,d) ((ipv4_t)(  ((uint32_t)(a)<<24) \
                                | ((uint32_t)(b)<<16) \
                                | ((uint32_t)(c)<< 8) \
                                |  (uint32_t)(d)     ))
```

7.4 Stack API Reference

7.5 ICMP (Ping)

```
icmp_clear_reply();
icmp_ping(IPV4(10,0,2,2), 1);          /* send echo request seq=1 */
/* poll until reply or timeout */
for (int i = 0; i < 10 && !icmp_got_reply(); i++) {
    net_stack_poll();
    /* sleep ~100 ms */
```

Function	Description
`net_stack_init(ip, gw, nm)`	Init; DHCP then static fallback
`net_stack_init_ex(ip, gw, nm, ms)`	Init with explicit DHCP timeout
`net_stack_is_up()`	1 if NIC found and stack is live
`net_stack_dhcp_used()`	1 if IP came from DHCP
`net_stack_poll()`	Process pending RX frames
`net_stack_tick()`	TCP retransmit / timeout tick
`net_stack_get_ip()`	Return assigned IPv4 address
`net_ip_to_str(ip, buf)`	Format `ipv4_t` as `"a.b.c.d"`
`net_str_to_ip(str, out)`	Parse `"a.b.c.d"` string

Table 7.1: TCP/IP stack top-level API (`kernel/net/stack.h`)

```
}
if (icmp_got_reply()) { /* success */ }
```

7.6 UDP

```
int s = udp_open(0);           /* ephemeral local port */
uint8_t msg[] = "hello";
udp_send(s, IPV4(10,0,2,2), 9, msg, sizeof(msg));

uint8_t buf[512]; ipv4_t src; uint16_t sport;
int n = udp_recv(s, buf, sizeof(buf), &src, &sport);
if (n > 0) { /* buf[0..n-1] contains datagram payload */ }
udp_close(s);
```

Up to 8 simultaneous UDP sockets. Each socket has a 2 KB receive ring buffer. `udp_recv()` is non-blocking and returns 0 if no datagram is waiting.

7.7 TCP

7.7.1 Client Connection

```
int sock = tcp_connect(IPV4(93,184,216,34), 80, 0);
while (tcp_state(sock) != TCP_ESTABLISHED) net_stack_poll();

uint8_t req[] = "GET␣/␣HTTP/1.0\r\nHost:␣example.com\r\n\r\n";
tcp_send(sock, req, sizeof(req) - 1);

uint8_t buf[512]; int n;
while ((n = tcp_recv(sock, buf, sizeof(buf))) > 0) {
    /* process buf[0..n-1] */
    net_stack_poll();
}
tcp_close(sock);
```

7.7.2 Passive Server

```
int lsock = tcp_listen(8080);
while (1) {
    net_stack_poll();
    int csock = tcp_accept(lsock);
    if (csock < 0) continue;
    /* handle connection ... */
    tcp_close(csock);
}
```

7.7.3 TCP API Reference

Function	Description
`tcp_connect(ip, port, lport)`	Active open; socket index or −1
`tcp_listen(port)`	Passive open; socket index or −1
`tcp_accept(lsock)`	Accept pending connection
`tcp_send(sock, data, len)`	Queue data; returns bytes queued
`tcp_recv(sock, buf, max)`	Non-blocking receive
`tcp_close(sock)`	Graceful close (sends FIN)
`tcp_state(sock)`	Returns `TcpState` enum
`tcp_tick()`	Retransmit / timeout (call from PIT)

Table 7.2: TCP API (`kernel/net/tcp.h`)

Up to 8 simultaneous TCP connections; 4 KB TX and 4 KB RX ring buffers per socket.

7.8 DHCP

`net_stack_init_ex()` invokes DHCP automatically. For direct use:

```
DhcpLease lease;
if (dhcp_request(&lease, 2000)) {
    /* lease.ip, .gateway, .netmask, .dns filled */
}
```

7.9 Polling Model

The stack has no dedicated network thread. RX is processed via two paths:

1. **IRQ path** — the NIC fires an interrupt; the handler calls `e1000_poll()` / `virtio_net_poll()`, which invokes the RX callback, which drives `eth_rx()` → `arp_rx()` / `ip_rx()` → TCP/UDP demux.
2. **Poll path** — `net_stack_poll()` is called from the main `hlt` loop and from `CMD_YIELD()` inside shell commands.

Both paths call `netdev_poll()` which is guarded by a per-driver reentrancy lock, making simultaneous calls safe.

Command	Description
`netstart`	Initialise stack from `S:net.conf`
`netstop`	Shut down stack, release DHCP lease
`ifconfig [dhcp | <ip> <gw>]`	Show or configure network
`ping <host> [count]`	Send ICMP echo requests
`route`	Display routing table and ARP cache
`nslookup <host> [server]`	Resolve hostname via DNS
`ntpd [server]`	Synchronise time via NTP
`netinfo`	Open network info window

Table 7.3: Network shell commands

File	Purpose	Default
`S:net.conf`	Mode (dhcp/static), IP, netmask, gateway, DNS	dhcp / 10.0.2.15/24
`S:ntp.conf`	NTP server hostname	pool.ntp.org
`S:timezone.conf`	IANA timezone name	Australia/Sydney

Table 7.4: Network configuration files

7.10 Shell Network Commands

7.10.1 Network Configuration Files

7.10.2 `bsdsocket.library` LVO Table

The `bsdsocket.library` ROM module exposes a standard AmigaOS BSD socket LVO table to M68k binaries. Descriptors 0–7 map to TCP sockets and 8–15 to UDP sockets.

LVO	Function	Arguments
-30	`socket`	domain/D0, type/D1, protocol/D2
-36	`bind`	fd/D0, sockaddr/A0, addrlen/D1
-42	`listen`	fd/D0, backlog/D1
-48	`accept`	fd/D0, sockaddr/A0, addrlen/A1
-54	`connect`	fd/D0, sockaddr/A0, addrlen/D1
-60	`send`	fd/D0, buf/A0, len/D1, flags/D2
-66	`sendto`	fd/D0, buf/A0, len/D1, flags/D2, addr/A1, addrlen/D3
-72	`recv`	fd/D0, buf/A0, len/D1, flags/D2
-78	`recvfrom`	fd/D0, buf/A0, len/D1, flags/D2, addr/A1, addrlen/A2
-84	`closesocket`	fd/D0
-96	`setsockopt`	fd/D0, level/D1, optname/D2, val/A0, optlen/D3
-102	`getsockopt`	fd/D0, level/D1, optname/D2, val/A0, optlen/A1
-108	`IoctlSocket`	fd/D0, req/D1, arg/A0
-132	`inet_addr`	str/A0
-138	`inet_ntoa`	addr/D0
-210	`gethostbyname`	name/A0

Table 7.5: `bsdsocket.library` LVO table

Chapter 8

Troubleshooting

8.1 GRUB: "Invalid arch-dependent ELF magic"

This error means GRUB received a file that is not a valid ELF64 executable. Ensure `uaos-kernel.elf` was produced by the full NASM + GCC + LD pipeline (not the Python mock generator used for structural validation only).

8.2 GRUB: "You need to load the kernel first"

GRUB could not find the `multiboot2` header within the first 32 KB of the binary. Verify the linker script places `.multiboot2` first and that the NASM source emits the section correctly.

8.3 Kernel halts immediately after boot

Enable serial output and check the UART log:

```
qemu-system-x86_64 -cdrom build/Ultimate_Amiga_OS.iso \
    -m 512M -boot d -serial stdio 2>&1 | tee boot.log
```

Appendix A

Amiga Memory Map Reference

See Table 2.1 in Chapter 2. The full OCS/ECS/AGA custom chip register map follows Commodore's *Hardware Reference Manual, 3rd Ed.*